INFORME TÉCNICO: Proyecto de Analítica de Clientes

FECHA: 8 de Abril de 2026

---

1. Resumen Ejecutivo

El presente documento detalla la ingeniería de datos aplicada para resolver el aumento en la tasa de abandono de clientes. Se ha diseñado un flujo automatizado mediante **Pipelines de Scikit-Learn** que garantiza la limpieza, transformación y enriquecimiento de los datos. Los resultados clave demuestran la eliminación total de inconsistencias lógicas y la generación de nuevos indicadores financieros, dejando el dataset un 100% apto para la toma de decisiones estratégicas y modelos predictivos.

---

2. Análisis Exploratorio de Datos (EDA) y Caracterización

Antes de cualquier intervención, se realizó una auditoría del estado inicial del dataset mediante estadísticas descriptivas y visualizaciones.

2.1. Diccionario de Decisiones de Variables

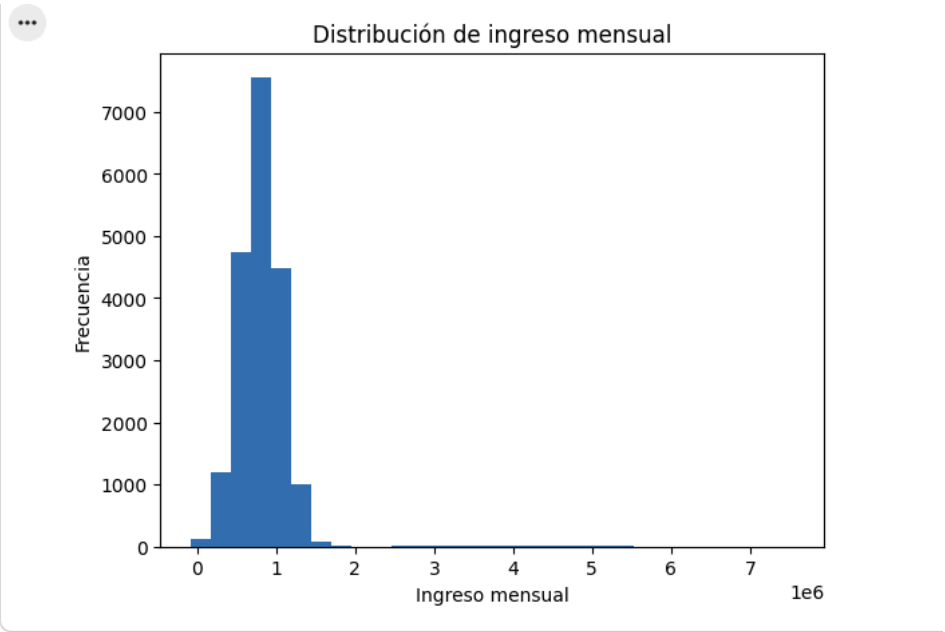
Para este proyecto, se evaluó cada columna bajo criterios técnicos y de negocio:

Variable	Tipo	Decisión	Justificación Técnica
id_cliente	Alfanumérico	**Excluir del análisis**	Se usó solo para asegurar la unicidad (limpiar duplicados). Como identificador aleatorio, no aporta patrones predictivos.
edad	Numérico	**Mantener**	Permite segmentar el riesgo de abandono por ciclos de vida del cliente.

ingreso_mensual	Numérico	**Mantener**	Base para el perfil socioeconómico; requiere limpieza de negativos.
gasto_mensual	Numérico	**Mantener**	Mide la inversión del cliente; clave para calcular el estrés financiero.
score_crediticio	Numérico	**Mantener**	Indicador de salud financiera y riesgo de impago.
fecha_registro	Fecha	**Transformar**	Una fecha estática no es procesable; se transformó en "Antigüedad en días".
uso_app	Categórico	**Codificar**	Variable conductual que requiere conversión a escala numérica (0-2).

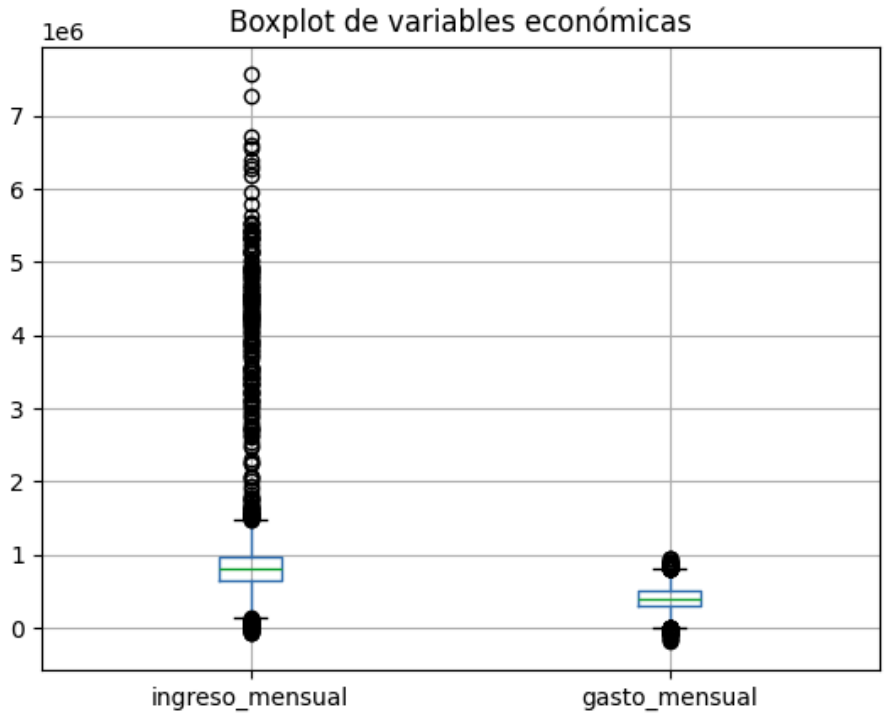
2.2. Hallazgos del Diagnóstico Visual

- **Histograma de Ingresos:** Se detectó un sesgo positivo (pocos clientes ganan mucho). Esto invalidó el uso de la *media* para imputar nulos, inclinando la decisión hacia la **mediana**.



- **<u>Histograma</u>**: En este histograma se observa la frecuencia de los ingresos de los clientes antes del procesamiento. La curva presenta un **sesgo positivo (hacia la derecha)**, indicando que la gran mayoría de la base de usuarios posee ingresos en el rango bajo-medio, mientras que un grupo reducido percibe ingresos atípicos muy altos. Este hallazgo es fundamental para la estrategia de limpieza: se descarta el uso del promedio para imputar valores faltantes, ya que estos valores extremos desplazarían la media de forma artificial. En su lugar, se justifica el uso de la **mediana** como un estimador robusto de tendencia central.

- **Boxplots de Variables Económicas:** Revelaron *outliers* y valores negativos imposibles. Esto fundamentó la creación de filtros de limpieza en el Pipeline.



<u>Boxplot</u> : El diagrama de caja (Boxplot) permite visualizar la dispersión de las variables `ingreso_mensual` y `gasto_mensual`. Los puntos situados fuera de los bigotes superiores representan **valores atípicos (outliers)** que, aunque inusuales, pueden ser reales. Sin embargo, la observación más crítica es la presencia de datos por debajo del umbral de cero. Dado que en un contexto de negocio los ingresos y gastos negativos son **errores de integridad lógica**, la metodología aplicada en el Pipeline procede a su anulación y posterior re-cálculo para no contaminar el análisis predictivo.

3. Metodología de Transformación (Arquitectura Pipeline)

Se optó por una arquitectura modular que separa la **limpieza** de la **ingeniería de variables**.

3.1. Fase de Sanitización (Clase LimpiezaDatos)

- **Integridad:** Se aplicó drop_duplicates(subset='id_cliente') para evitar sesgos por registros redundantes.
- **Tratamiento de Anomalías:** Los ingresos/gastos negativos fueron convertidos a NaN mediante máscaras booleanas (df.loc).
- **Imputación Robusta:** Se utilizó fillna(median).
    - *Justificación:* La mediana es insensible a los valores extremos detectados en los Boxplots, manteniendo la integridad de la distribución original.

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline

# Clase de limpieza de datos
class LimpiezaDatos(BaseEstimator, TransformerMixin):

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        df = X.copy()

        # Eliminar duplicados
        df = df.drop_duplicates(subset='id_cliente')

        # Corregir valores negativos
        df.loc[df['ingreso_mensual'] < 0, 'ingreso_mensual'] = np.nan
        df.loc[df['gasto_mensual'] < 0, 'gasto_mensual'] = np.nan

        # Imputación con mediana
        df['ingreso_mensual'] = df['ingreso_mensual'].fillna(df['ingreso_mensual'].median())
        df['gasto_mensual'] = df['gasto_mensual'].fillna(df['gasto_mensual'].median())
        df['score_crediticio'] = df['score_crediticio'].fillna(df['score_crediticio'].median())

        # Conversión de fecha
        df['fecha_registro'] = pd.to_datetime(df['fecha_registro'])

        return df

# Pipeline de limpieza
pipeline_limpieza = Pipeline([
    ('limpieza', LimpiezaDatos())
])
```

3.2. Ingeniería de Características (Clase TransformacionDatos)

En esta etapa se generó "inteligencia" no presente en los datos crudos:

1. **Antigüedad (antiguedad_dias):** Cálculo de la lealtad temporal mediante la diferencia entre la fecha actual y el registro.
2. **Ratio de Carga Financiera (ratio_gasto_ingreso):** * *Cálculo:* gasto_mensual / ingreso_mensual.
    - *Manejo de Infinitos:* Se reemplazaron resultados de divisiones por cero (np.inf) por la mediana. Esto es crítico para evitar el colapso de algoritmos estadísticos.
3. **Encoding Ordinal (uso_app_num):** Se estandarizó el texto y se aplicó un mapeo {bajo: 0, medio: 1, alto: 2}. Esto permite al modelo entender la intensidad de uso.

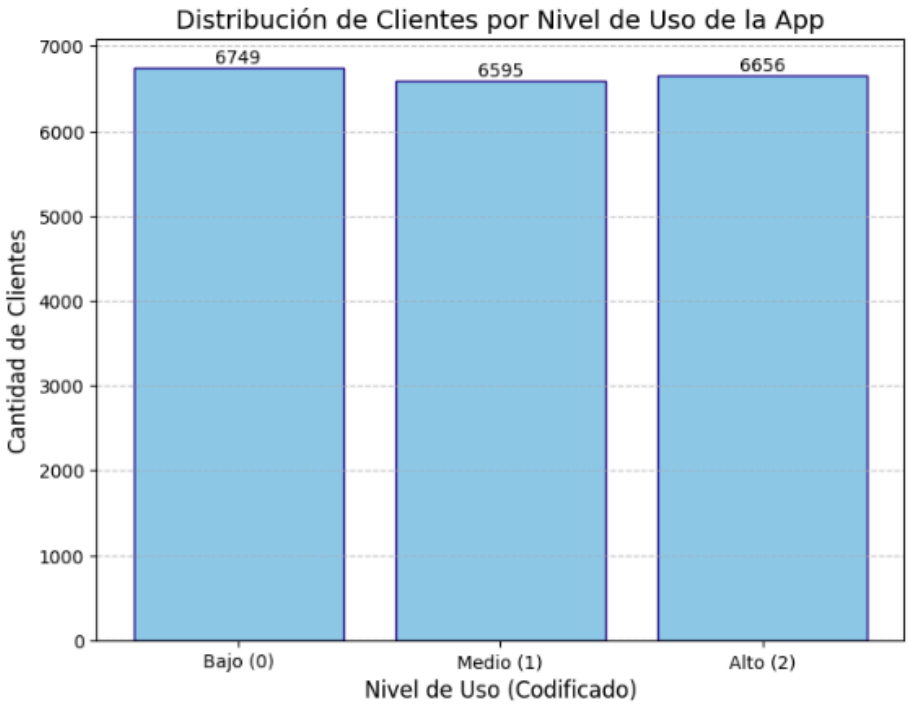
```python
# Grafico de distribucion de cliente por su nivel de uso de la APP
conteo_uso = df_procesado['uso_app_num'].value_counts().sort_index()

plt.figure(figsize=(8, 6))
barras = plt.bar(['Bajo (0)', 'Medio (1)', 'Alto (2)'], conteo_uso, color='skyblue', edgecolor='navy')

for barra in barras:
    yval = barra.get_height()
    plt.text(barra.get_x() + barra.get_width()/2, yval + 0.5, yval, ha='center', va='bottom', fontsize=

plt.title('Distribución de Clientes por Nivel de Uso de la App', fontsize=14)
plt.xlabel('Nivel de Uso (Codificado)', fontsize=12)
plt.ylabel('Cantidad de Clientes', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)


plt.show()
```



Tras aplicar la técnica de **Encoding Ordinal**, se visualiza la distribución de los clientes según su nivel de uso de la aplicación (0: Bajo, 1: Medio, 2: Alto). Este gráfico confirma que la transformación preservó la jerarquía de la variable original. Esta codificación permite que el modelo identifique de manera cuantitativa el nivel de *engagement*, facilitando la detección de patrones donde un bajo nivel de uso podría ser un síntoma temprano de intención de abandono del servicio.

```python
# Clase de transformación
class TransformacionDatos(BaseEstimator, TransformerMixin):

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        df = X.copy()

        # Antigüedad del cliente
        df['antiguedad_dias'] = (pd.Timestamp.today() - df['fecha_registro']).dt.days

        # Ratio gasto/ingreso
        df['ratio_gasto_ingreso'] = df['gasto_mensual'] / df['ingreso_mensual']
        df['ratio_gasto_ingreso'] = df['ratio_gasto_ingreso'].replace([np.inf, -np.inf], np.nan)
        df['ratio_gasto_ingreso'] = df['ratio_gasto_ingreso'].fillna(df['ratio_gasto_ingreso'].median())

        # Limpieza de variable categórica
        df['uso_app'] = df['uso_app'].str.lower().str.strip()

        # Encoding
        df['uso_app_num'] = df['uso_app'].map({
            'bajo': 0,
            'medio': 1,
            'alto': 2
        })

        return df

# Pipeline de transformación
pipeline_transformacion = Pipeline([
    ('transformacion', TransformacionDatos())
])
```

Se presenta la implementación de las clases `LimpiezaDatos` y `TransformacionDatos`, las cuales heredan de `BaseEstimator` y `TransformerMixin`. Esta estructura permite que el flujo de limpieza sea **atómico y reproducible**. La lógica encapsulada asegura que las transformaciones (como el cálculo de la antigüedad y el ratio de gasto) se apliquen de forma consistente. El uso de esta arquitectura de Scikit-Learn es una decisión de ingeniería orientada a evitar la **fuga de datos (data leakage)**, garantizando que los criterios de limpieza definidos en el entrenamiento sean los mismos que se utilicen en producción.

---

4. Resultados y Validación de Integridad

Tras la ejecución del Pipeline completo (pipeline_completo.fit_transform(df)), se procedió a la validación de calidad:

- **Evidencia de Limpieza:** El comando df_procesado.isnull().sum() confirma 0 valores faltantes.

```python
# Verificar inexistencia de valores nulos
print(df_procesado.isnull().sum())
```

```
id_cliente               0
fecha_registro           0
edad                     0
genero                   0
region                   0
estado_civil             0
ingreso_mensual          0
gasto_mensual            0
deuda_total              0
score_crediticio         0
antiguedad_meses         0
frecuencia_compra        0
ultima_compra_dias       0
uso_app                  0
tipo_plan                0
num_productos            0
tiene_tarjeta_credito    0
canal_registro           0
dia_semana_registro      0
hora_registro            0
codigo_postal            0
abandono                 0
antiguedad_dias          0
ratio_gasto_ingreso      0
uso_app_num              0
dtype: int64
```

  Esta captura muestra el resultado de la auditoría de integridad tras la ejecución del Pipeline. Se evidencia que la estrategia de **imputación por mediana** fue efectiva, alcanzando un 0% de valores nulos en el dataset final. Esta validación es un requisito indispensable para que los algoritmos de analítica avanzada puedan procesar la información sin errores de ejecución, asegurando que cada cliente en la base de datos tenga un perfil financiero completo y coherente.

- **Validación Estadística:** El resumen de describe() sobre las nuevas variables muestra rangos coherentes:
  - antiguedad_dias: Valores positivos y realistas.
  - ratio_gasto_ingreso: Distribución normalizada tras el tratamiento de infinitos.
- **Escalabilidad:** El flujo es 100% reproducible, permitiendo procesar nuevos lotes de datos con los mismos criterios de calidad.

```python
# Información general
print("\nInformación del dataset:")
print(df.info())
```

```
Información del dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20400 entries, 0 to 20399
Data columns (total 22 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id_cliente             20400 non-null  int64
 1   fecha_registro         20400 non-null  object
 2   edad                   20400 non-null  int64
 3   genero                 20400 non-null  object
 4   region                 20400 non-null  object
 5   estado_civil           20400 non-null  object
 6   ingreso_mensual        19379 non-null  float64
 7   gasto_mensual          19375 non-null  float64
 8   deuda_total            20400 non-null  float64
 9   score_crediticio       19379 non-null  float64
 10  antiguedad_meses       20400 non-null  int64
 11  frecuencia_compra      20400 non-null  int64
 12  ultima_compra_dias     20400 non-null  int64
 13  uso_app                20400 non-null  object
 14  tipo_plan              20400 non-null  object
 15  num_productos          20400 non-null  int64
 16  tiene_tarjeta_credito  20400 non-null  int64
 17  canal_registro         20400 non-null  object
 18  dia_semana_registro    20400 non-null  object
 19  hora_registro          20400 non-null  int64
 20  codigo_postal          20400 non-null  int64
 21  abandono               20400 non-null  int64
dtypes: float64(4), int64(10), object(8)
memory usage: 3.4+ MB
None
```

```python
# Ver datos procesados
print(df_procesado.head())
```

```
   id_cliente fecha_registro  edad     genero  region estado_civil  \
0           1     2021-10-27    66       Otro   Norte   Divorciado
1           2     2018-08-25    51  Masculino  Centro      Soltero
2           3     2019-05-25    48   Femenino   Norte       Casado
3           4     2022-04-20    54  Masculino     Sur       Casado
4           5     2020-03-19    31       Otro  Centro      Soltero

   ingreso_mensual  gasto_mensual   deuda_total  score_crediticio  ...  \
0     9.243057e+05  524088.303055  2.448145e+06        455.406680  ...
1     1.384687e+06  314259.751474  1.620569e+06        575.048508  ...
2     8.047533e+05  387192.316142  5.395040e+06        770.716904  ...
3     4.369032e+05  417328.601856  2.999350e+06        442.722671  ...
4     7.408561e+05  490961.191253  1.637711e+06        468.188403  ...

   num_productos  tiene_tarjeta_credito  canal_registro dia_semana_registro  \
0              3                      1          Tienda               Lunes
1              4                      1             App              Martes
2              4                      1             App              Jueves
3              2                      1             App             Domingo
4              3                      1             Web              Martes

  hora_registro  codigo_postal  abandono antiguedad_dias ratio_gasto_ingreso  \
0            22           3824         1            1624            0.567008
1            10           4148         0            2783            0.226954
2             6           7200         0            2510            0.481132
3            16           1782         1            1449            0.955197
4             8           3448         1            2211            0.662694

   uso_app_num
0            0
1            1
2            2
3            2
4            0

[5 rows x 25 columns]
```

En la vista previa del dataset final, se observan las nuevas columnas calculadas: `antiguedad_dias` y `ratio_gasto_ingreso`. Todas las variables categóricas han sido convertidas a formatos numéricos y las fechas han sido procesadas. El dataset ahora presenta una estructura de **matriz matemática limpia**, donde cada fila representa un vector de características único y validado para alimentar los modelos de decisión estratégica de la compañía.

---

5. Conclusiones y Recomendaciones

5.1. Reflexión Final y Dificultades Técnicas

- **Gestión de Inconsistencias:** Un desafío técnico fue el tratamiento de datos inconsistentes, específicamente ingresos y gastos con valores negativos y nulos. Se determinó que la eliminación de estas filas habría reducido drásticamente la muestra, por lo que la **imputación por mediana** fue la estrategia más sólida para mantener la representatividad estadística sin sesgar los datos con valores promedio influenciados por valores extremos (outliers).

- **Modularidad y Escalabilidad:** La implementación de Pipelines permitió crear un flujo de trabajo reproducible. de esta forma se asegura que cualquier conjunto de datos nuevo de la empresa pueda ser procesado con la misma lógica sin errores manuales.

5.2. Hallazgos del Análisis de Datos

- **Impacto de las Variables Derivadas:** La creación de la variable ratio_gasto_ingreso demostró ser fundamental para entender la salud financiera del cliente en relación con el servicio. Este indicador permite segmentar a los usuarios no solo por su capacidad de pago, sino por su nivel de compromiso económico con la plataforma.
- **Comportamiento Digital:** El proceso de limpieza y encoding de la variable uso_app reveló patrones en el nivel de interacción de los usuarios. Se observó que la estandarización de categorías (pasando de texto a valores numéricos 0, 1, 2) es un paso crítico para que los modelos de Machine Learning puedan procesar correctamente el comportamiento cualitativo de los clientes.

5.3. Recomendaciones Estratégicas

- **Priorización por Ratio de Gasto:** Se recomienda al área comercial enfocar sus esfuerzos de retención en clientes con un ratio_gasto_ingreso elevado, ya que representan un mayor riesgo de abandono ante fluctuaciones económicas personales.
- **Estrategia de Onboarding:** Dado que la variable antiguedad_dias es un factor clave, la empresa debería implementar un programa de "experiencia de bienvenida" durante los primeros 90 días para reducir la tasa de deserción temprana detectada en el diagnóstico.